Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЕТ
к лабораторной работе №7
на тему:
«ЗАЩИТА ПО ОТ НЕСАНКЦИОНИРОВАННОГО ИСПОЛЬЗОВАНИЯ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГЕРЧИК Артём Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Краткие теоритичесие сведения 4
3 Выполнение работы 5
Вывод 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Провести обфускацию заданной программы.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Обфускация – это процесс преобразования исполняемого кода с целью затруднения его анализа и понимания человеком, а также для уменьшения размера приложения и повышения производительности.

В экосистеме Android основными инструментами обфускации являются ProGuard и R8 (последний используется по умолчанию в современных версиях Android Gradle Plugin). Обфускация включает в себя:

– удаление неиспользуемых классов, полей и методов;

– переименование оставшихся классов, методов и полей в короткие бессмысленные идентификаторы;

– оптимизацию байт-кода (например, инлайнинг, упрощение выражений);

– сжатие ресурсов (shrinkResources) – удаление неиспользуемых ресурсов из проекта.

Включается обфускация в конфигурации сборки release путём установки флагов isMinifyEnabled = true и isShrinkResources = true. При этом используются стандартные правила ProGuard из файла proguard-android-optimize.txt и дополнительные правила из файла proguard-rules.pro, которые учитывают особенности используемых библиотек.

Если количество методов в приложении превышает лимит в 64K (предел для одного dex-файла), Android автоматически включает multidex, создавая несколько dex-файлов. Обфускация может сократить число методов и позволить уместить весь код в один dex-файл.

3 ВЫПОЛНЕНИЕ РАБОТЫ

В рамках лабораторной работы была проведена обфускация Android приложения. В файле build.gradle модуля приложения в блоке buildTypes для сборки release заданы необходимые параметры, описанные выше.

Для анализа результатов были собраны две версии APK: release без обфускации и release с обфускацией. Структура и размеры APK исследовались с помощью встроенного инструмента Android Studio – APK Analyzer.

На рисунке 3.1 представлен анализ APK до обфускации. Общий размер APK составлял 16,4 МБ, скачиваемый размер – 15,3 МБ. Основной объём занимали dex-файлы: classes.dex – 16,2 МБ, classes2.dex – 6,6 МБ, classes3.dex – 4,4 МБ, classes4.dex – 377 КБ.

APK size: **16,4 MB**, Download Size: **15,3 MB**

File	Size	Download Size	% of Download Size
	16,2 MB	15,4 MB	100%
classes.dex	6,6 MB	6,5 MB	42,3%
classes2.dex	4,4 MB	4,3 MB	28,1%
classes3.dex	3,3 MB	3,3 MB	21,6%
res	628,3 KB	635,6 KB	4%
classes4.dex	377 KB	376,1 KB	2,4%
resources.arsc	675,5 KB	130,3 KB	0,8%
okhttp3	40,6 KB	40,6 KB	0,3%
META-INF	30 KB	32 KB	0,2%
lib	245,9 KB	31,9 KB	0,2%
assets	13,1 KB	12,6 KB	0,1%
kotlin	12,1 KB	12,1 KB	0,1%
AndroidManifest.xml	2,4 KB	2,4 KB	0%
DebugProbesKt.bin	780 B	798 B	0%
kotlin-tooling-metadata.json	282 B	300 B	0%

Рисунок 3.1 – APK-файл Android приложения до обфускации

На рисунке 3.2 – после применения обфускации и сжатия ресурсов размер APK сократился до 3,5 МБ (скачиваемый – 2,7 МБ). Все классы поместились в один classes.dex размером 3,4 МБ. Размер библиотеки okhttp3 уменьшился до 553 КБ (скачиваемый – 106 КБ) благодаря удалению неиспользуемых частей. Общее сжатие APK составило 4,7 раза, а dex-файлов – более чем в 8 раз (с 27,6 МБ до 3,4 МБ).

APK size: **3,5 MB**, Download Size: **2,7 MB**

File	Size	Download Size	% of Download Size
	3,4 MB	2,7 MB	100%
classes.dex	1,9 MB	1,9 MB	71,4%
res	556,8 KB	560,3 KB	20,2%
resources.arsc	553,4 KB	106,6 KB	3,8%
okhttp3	40,6 KB	40,6 KB	1,5%
META-INF	30,1 KB	32 KB	1,2%
lib	245,9 KB	31,9 KB	1,2%
kotlin	12,1 KB	12,1 KB	0,4%
assets	6,8 KB	6,8 KB	0,2%
AndroidManifest.xml	2,4 KB	2,4 KB	0,1%
DebugProbesKt.bin	780 B	798 B	0%
kotlin-tooling-metadata.json	282 B	300 B	0%

Рисунок 3.2 – APK-файл Android приложения после обфускации

На рисунке 3.3 показаны имена папок, классов, методов заменённые на короткие идентификаторы (например, u2, q, b0, t). Для класса u1 приведены его методы, также переименованные.

Class	Defined Methods	Referenced Methods	Size
u2	512	512	92,2 KB
q	417	417	62,5 KB
b0	394	394	51,9 KB
t	380	380	51,3 KB
c2	19	19	1,7 KB
d	18	18	3,3 KB
u1	18	18	3,8 KB
<init>(f0.i0,t.u1,java.lang.String)	1	1	234 B
void a(java.lang.Object,f1.t,int)	1	1	468 B
long b()	1	1	156 B
void c()	1	1	124 B
boolean d()	1	1	132 B
long e()	1	1	74 B

Рисунок 3.3 – Результат обфускации Android приложения

ВЫВОД

В результате выполнения лабораторной работы была проведена обфускация Android-приложения с использованием встроенных средств Gradle (R8/ProGuard). Эффективность обфускации подтверждается значительным уменьшением размера APK (с 16,4 МБ до 3,5 МБ), сокращением количества dex-файлов с четырёх до одного и переименованием всех классов и методов в короткие неинформативные идентификаторы. Это не только снижает трафик при загрузке приложения и экономит место на устройстве пользователя, но и затрудняет обратную разработку кода, защищая интеллектуальную собственность разработчика. Дополнительно включённое сжатие ресурсов позволило удалить неиспользуемые ресурсы, что также внесло вклад в уменьшение размера.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)

    alias(libs.plugins.google.devtools.ksp)

    // Dagger - Hilt
    alias(libs.plugins.dagger.hilt.android)
}

android {
    namespace = "com.artkotlindev.reminderbirthday_calendar"
    compileSdk = 36

    signingConfigs {
        create("release") {
            storeFile = file(project.property("RELEASE_STORE_FILE") as String)
            storePassword  =  project.property("RELEASE_STORE_PASSWORD")  as
String
            keyAlias = project.property("RELEASE_KEY_ALIAS") as String
            keyPassword = project.property("RELEASE_KEY_PASSWORD") as String
        }
    }

    defaultConfig {
        applicationId = "com.artkotlindev.reminderbirthday_calendar"
        minSdk = 26
        targetSdk = 36
        versionCode = 1054
        versionName = "1.0.5.4"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"

        signingConfig = signingConfigs.getByName("release")
    }

    androidResources {
        localeFilters.addAll(listOf("en", "ru", "be"))
    }

    bundle {
        language {
            enableSplit = false
        }
    }

    buildTypes {
        release {
            // Enables code-related app optimization.
            isMinifyEnabled = true

            // Enables resource shrinking.
            isShrinkResources = true
```

```
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
    kotlin {
        compilerOptions {
            jvmTarget.set(org.jetbrains.kotlin.gradle.dsl.JvmTarget.JVM_11)
        }
    }
    buildFeatures {
        compose = true
    }
}

dependencies {
    implementation(project(":data"))
    implementation(project(":domain"))

    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.ui)
    implementation(libs.androidx.ui.graphics)
    implementation(libs.androidx.ui.tooling.preview)
    implementation(libs.androidx.material3)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    debugImplementation(libs.androidx.ui.tooling)
    debugImplementation(libs.androidx.ui.test.manifest)

    // icons extended
    implementation(libs.androidx.compose.material.icons.extended)

    // RoomDb
    implementation(libs.androidx.room.ktx)
    implementation(libs.androidx.room.runtime)
    ksp(libs.androidx.room.compiler)

    // Dagger - Hilt
    implementation(libs.dagger.hilt.android)
    ksp(libs.dagger.hilt.android.compiler)

    // Dagger - Hilt (navigation)
    implementation (libs.androidx.hilt.navigation.compose)
    ksp(libs.androidx.hilt.compiler)

    // Jetpack Compose permissions
    implementation(libs.accompanist.permissions)

    // For pick photos from gallery
    implementation(libs.coil.kt.compose)
}
```